Code Explanation

Data Processing Pipeline Overview

The given code is a data processing pipeline written in Python using the PySpark library. It reads customer and order data from CSV files, cleans and transforms the data, creates and updates a Slowly Changing Dimension (SCD) Type 2 table, and finally creates a customer aggregate spend table.

Step 1: Initialize Spark Session and Read Source Data
The code starts by initializing a Spark Session and reading customer and order data from CSV files using the `read_source_data` function. This function defines the schema for the customer and order data and reads the data into DataFrames.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder
    .appName("Customer Order Processing")
    .getOrCreate()

def read_source_data(spark, customer_path, order_path):
    customer_schema = StructType([
        StructField("CustId", StringType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])

    order_schema = StructType([
        StructField("OrderId", StringType(), True),
        StructField("ItemName", StringType(), True),
        StructField("PricePerUnit", DoubleType(), True),
        StructField("Qty", IntegerType(), True),
        StructField("Date", DateType(), True),
        StructField("CustId", StringType(), True)
    ])

    customer_df = spark.read.format("csv")
        .option("header", "true")
        .schema(customer_schema)
        .load(customer_path)

    order_df = spark.read.format("csv")
        .option("header", "true")
        .schema(order_schema)
        .load(order_path)

    return customer_df, order_df
```

Step 2: Clean and Transform Data
The `clean_and_transform_data` function removes null values and duplicates from the customer and order data and adds a `TotalAmount` column to the order data.

```
def clean_and_transform_data(customer_df, order_df):
    clean_customer_df = customer_df.filter(
        col("CustId").isNotNull() &
```

```
        col("Name").isNotNull() &
        col("EmailId").isNotNull() &
        col("Region").isNotNull()
    ).dropDuplicates()

    clean_order_df = order_df.filter(
        col("OrderId").isNotNull() &
        col("ItemName").isNotNull() &
        col("PricePerUnit").isNotNull() &
        col("Qty").isNotNull() &
        col("Date").isNotNull() &
        col("CustId").isNotNull()
    ).dropDuplicates()
     .withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))

    return clean_customer_df, clean_order_df
```

Step 3: Create and Update SCD Type 2 Table

The `create_and_update_scd_type2_table` function joins the customer and order data, checks if the target table exists, and creates or updates the SCD Type 2 table accordingly.

```
def create_and_update_scd_type2_table(spark, customer_df, order_df,
catalog, schema):
    joined_df = customer_df.join(order_df, "CustId", "inner").select
(
        customer_df["CustId"],
        customer_df["Name"],
        customer_df["EmailId"],
        customer_df["Region"],
        order_df["OrderId"],
        order_df["ItemName"],
        order_df["PricePerUnit"],
        order_df["Qty"],
        order_df["Date"],
        order_df["TotalAmount"]
    )

    table_exists = False
    try:
        spark.sql(f"DESCRIBE TABLE {catalog}.{schema}.ordersummary")
        table_exists = True
    except:
        table_exists = False

    if not table_exists:
        # Create the table
        joined_df = joined_df.withColumn("IsActive", lit(True))
                             .withColumn("StartDate", current_timestam
p())
                             .withColumn("EndDate", lit(None).cast(Tim
estampType()))

        joined_df.write.format("delta")
            .mode("overwrite")
            .option("overwriteSchema", "true")
            .saveAsTable(f"{catalog}.{schema}.ordersummary")
```

```
    else:
        # Update the existing SCD Type 2 table
        delta_table = DeltaTable.forName(spark, f"{catalog}.{schema}
.ordersummary")

        update_df = joined_df.withColumn("mergeKey", expr("CONCAT(Cu
stId, '|', OrderId)"))
        target_df = delta_table.toDF().withColumn("mergeKey", expr("
CONCAT(CustId, '|', OrderId)"))

        delta_table.alias("target").merge(
            update_df.alias("source"),
            "target.mergeKey = source.mergeKey"
        ).whenMatchedAndExpr(
            # Update condition
        ).updateExpr(
            # Update expression
        ).whenNotMatchedInsertExpr(
            # Insert expression
        ).execute()

        # Insert new records for updated rows
        updated_records = delta_table.toDF().filter(
            (col("IsActive") == False) & (col("EndDate") == current_
timestamp())
        )

        if updated_records.count() > 0:
            new_active_records = updated_records.select(
                "CustId", "Name", "EmailId", "Region", "OrderId", "I
temName",
                "PricePerUnit", "Qty", "Date", "TotalAmount"
            ).withColumn("IsActive", lit(True))
             .withColumn("StartDate", current_timestamp())
             .withColumn("EndDate", lit(None).cast(TimestampType()))

            new_active_records.write.format("delta")
                .mode("append")
                .saveAsTable(f"{catalog}.{schema}.ordersummary")
```

Step 4: Create Customer Aggregate Spend Table

The `create_customer_aggregate_spend` function aggregates data from the `ordersummary` table and creates a customer aggregate spend table.

```
def create_customer_aggregate_spend(spark, catalog, schema):
    aggregate_df = spark.sql(f"""
        SELECT Name, SUM(TotalAmount) as TotalAmount, Date
        FROM {catalog}.{schema}.ordersummary
        WHERE IsActive = true
        GROUP BY Name, Date
    """)

    aggregate_df.write.format("delta")
        .mode("overwrite")
        .saveAsTable(f"{catalog}.{schema}.customeraggregatespend")
```

Step 5: Main Function
The `main` function orchestrates the entire data processing pipeline.

```
def main():
    spark = SparkSession.builder
        .appName("Customer Order Processing")
        .getOrCreate()

    customer_path = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/c
ustomerdata"
    order_path = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orde
rdata"

    catalog = "gen_ai_poc_databrickscoe"
    schema = "sdlc_wizard"

    customer_df, order_df = read_source_data(spark, customer_path, o
rder_path)
    clean_customer_df, clean_order_df = clean_and_transform_data(cus
tomer_df, order_df)
    create_and_update_scd_type2_table(spark, clean_customer_df, clea
n_order_df, catalog, schema)
    create_customer_aggregate_spend(spark, catalog, schema)

if __name__ == "__main__":
    main()
```